Build Task — Project 3: LangGraph App — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python — how a real codebase would actually write it).

This Build Task **is** Project 3 — the full step-by-step build lives in Project 3's READING_README. The solution below is the same graph at its smallest, in one folder, so you can see every piece working together first.

- Hint 1 — The shape, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — The shape, and the exact pieces

Basic Version

One graph, made of pieces this document's exercises already practiced:

```
question -> route_search ---skip---> reason_node -> approval_node -> finish
                 |                        ^
               search                     | reason
                 v                        |
           search_node -> has_grounding --no_grounding--> no_grounding_node
```

- **The entry decision** (`route_search` , wired with `set_conditional_entry_point`): the Intermediate exercise.
- **The search** (Doc08's `retrieve(collection, query)` through `kb.py`): the Basic exercise.
- **"Nothing relevant found"** (`has_grounding` with `MAX_DISTANCE`): the Edge cases exercise.
- **The approval pause** (`interrupt()` + `Command(resume=...)` + a checkpointer): the Real-world exercise.
- **Showing progress:** Doc07's `stream_mode="updates"` — print each node's name as it finishes.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

On top of that:

- **A visible path:** a `path: Annotated[list[str], operator.add]` field; every node adds one short entry (`"searched"` , `"no grounding"` , `"approved"`), so the final state says exactly what happened.
- **`build_graph(checkpointer=None)`** — Doc09's Build Task, so a test can pass its own.
- **`test_project3.py`** — the README's 5 Test Cases, each printing `OK` or `WRONG` .
- **Retry on search** — the Failure exercise's `search_with_retry()` .
- **Who approved, and when** — the Real-world exercise's structured resume payload.
- **A validated state** — Pydantic's `BaseModel` (Doc04) instead of `TypedDict` . Watch out: nodes then get an **object**, so they read `state.task` , not `state["task"]` .

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state.py:  task, sources, draft, approved, answer

nodes.py:
    route_search(state):  "search" if a keyword is in the task, else "skip"
    search_node(state):   {"sources": retrieve(collection, state["task"])}
    has_grounding(state): "no_grounding" if no sources, or the best (smallest)
                          score > MAX_DISTANCE; else "reason"
    reason_node(state):   answer from the sources' text, or directly if none
    no_grounding_node:    the honest "I don't have grounding" answer
    approval_node(state): decision = interrupt({"draft": ...})
    finish_node(state):   the draft if approved, else "Not approved"

graph.py:  build_graph() -- the diagram above, compiled with MemorySaver()

main.py:   stream the run (print each node name), and if it paused,
           show the draft, ask yes/no, resume with Command(resume=...)
```

Here's almost the whole `main.py` — fill in the resume part:

```
def main():
    graph = build_graph()
    config = {"configurable": {"thread_id": "run-1"}}
    task = input("Your question: ")
    show_steps(graph, {"task": task}, config)

    paused = graph.get_state(config)
    if paused.next:
        print("Draft:", paused.values["draft"])
        # your turn: ask yes/no, then show_steps(graph,
        # Command(resume=...), config)
        ...

    print("Answer:", graph.get_state(config).values["answer"])
```

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

```
state.py:   + path: Annotated[list[str], operator.add]
nodes.py:   each node returns its path entry too, e.g. {"path": ["searched"]}
graph.py:   build_graph(checkpointer=None)
test_project3.py:
    run(graph, task, thread_id, decision) -> final state values
    case 1 search + found + approved   case 2 no search
    case 3 nothing relevant            case 4 approved -> answer == draft
    case 5 rejected -> "Not approved"

Approach 2: search_node calls search_with_retry(); the resume value is
            {"decision": ..., "by": ..., "at": ...}
Approach 3: ProjectState(BaseModel) with defaults; nodes use state.task
```

Difference between Basic and Intermediate: Basic wires the five practiced pieces into one working graph. Intermediate makes it checkable (a path, OK/WRONG tests), sturdier (retries, who-and-when), and stricter (a validated state).

Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Read both depths below — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Your files:

```
10_agent_workflows/practice/build_task/
├── docs/                the coffee documents (Doc08)
├── chunking.py ... retriever.py   Doc08's Build Task files, unchanged
├── graph.py             build_graph()
├── kb.py                builds the knowledge base once (the exercises' file)
├── main.py              runs a question, shows each step, asks for approval
├── nodes.py             route_search, search_node, has_grounding,
│                        reason_node, no_grounding_node, approval_node,
│                        finish_node
├── state.py             ProjectState
└── test_project3.py     Intermediate: the 5 Test Cases
```

Basic Version

Approach 1 — all five pieces in one graph

Story — `state.py` / `nodes.py` / `graph.py` / `main.py`: this is Project 3 at its smallest — every exercise above (search as a tool, the entry decision, the empty-search check, the approval pause, the search failure) meets in one graph. **If not:** Project 3 would be the first place these five pieces ever had to work together.

```
# state.py
from typing import TypedDict


class ProjectState(TypedDict):
    task: str
    sources: list
    draft: str
    approved: bool
    answer: str
```

```
# nodes.py
from langchain_openai import ChatOpenAI
from langgraph.types import interrupt

from kb import collection
from retriever import retrieve
from state import ProjectState

KEYWORDS = ["coffee", "brew", "grind", "water", "beans", "according to"]
MAX_DISTANCE = 1.2


def route_search(state: ProjectState) -> str:
    task = state["task"].lower()
    for keyword in KEYWORDS:
        if keyword in task:
            return "search"
    return "skip"


def search_node(state: ProjectState) -> dict:
    return {"sources": retrieve(collection, state["task"])}


def has_grounding(state: ProjectState) -> str:
    sources = state["sources"]
    if not sources:
        return "no_grounding"
    best = sources[0]["score"]
    for source in sources:
        if source["score"] < best:
            best = source["score"]
    if best > MAX_DISTANCE:
        return "no_grounding"
    return "reason"


def reason_node(state: ProjectState) -> dict:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    if not state.get("sources"):
        return {"draft": model.invoke(state["task"]).content}
    pieces = []
    for source in state["sources"]:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = "Context:\n" + context + "\n\nQuestion: " + state["task"]
    return {"draft": model.invoke(prompt).content}
```

```
def no_grounding_node(state: ProjectState) -> dict:
    return {"answer": "I don't have grounding for this in my documents."}


def approval_node(state: ProjectState) -> dict:
    decision = interrupt({"draft": state["draft"]})
    return {"approved": decision == "approved"}


def finish_node(state: ProjectState) -> dict:
    if state["approved"]:
        return {"answer": state["draft"]}
    return {"answer": "Not approved -- nothing was sent."}
```

```
# graph.py
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, StateGraph

from nodes import (
    approval_node,
    finish_node,
    has_grounding,
    no_grounding_node,
    reason_node,
    route_search,
    search_node,
)
from state import ProjectState


def build_graph():
    builder = StateGraph(ProjectState)
    builder.add_node("search_node", search_node)
    builder.add_node("reason_node", reason_node)
    builder.add_node("no_grounding_node", no_grounding_node)
    builder.add_node("approval_node", approval_node)
    builder.add_node("finish_node", finish_node)

    builder.set_conditional_entry_point(
        route_search, {"search": "search_node", "skip": "reason_node"}
    )
    builder.add_conditional_edges(
        "search_node", has_grounding,
        {"reason": "reason_node", "no_grounding": "no_grounding_node"},
```

```
    )
    builder.add_edge("reason_node", "approval_node")
    builder.add_edge("approval_node", "finish_node")
    builder.add_edge("no_grounding_node", END)
    builder.add_edge("finish_node", END)
    return builder.compile(checkpointer=MemorySaver())
```

```
# main.py
from langgraph.types import Command

from graph import build_graph


def show_steps(graph, graph_input, config):
    for update in graph.stream(graph_input, config, stream_mode="updates"):
        for node_name in update:
            print("  step:", node_name)


def main():
    graph = build_graph()
    config = {"configurable": {"thread_id": "run-1"}}
    task = input("Your question: ")
    show_steps(graph, {"task": task}, config)

    paused = graph.get_state(config)
    if paused.next:
        print("Draft:", paused.values["draft"])
        if input("Approve? (yes/no): ").strip().lower() == "yes":
            decision = "approved"
        else:
            decision = "rejected"
        show_steps(graph, Command(resume=decision), config)

    print("Answer:", graph.get_state(config).values["answer"])


if __name__ == "__main__":
    main()
```

Run it: `python main.py`, then answer the questions. **Expected session** (the wording varies):

```
Your question: How hot should the water be when brewing?
  step: search_node
  step: reason_node
```

```
  step: __interrupt__
Draft: Water for brewing coffee should be between 195 and 205 degrees F.
Approve? (yes/no): yes
  step: approval_node
  step: finish_node
Answer: Water for brewing coffee should be between 195 and 205 degrees F.
```

`__interrupt__` is the stream telling you the run paused. Try "What is 2 + 2?" (no `search_node` step), an off-topic coffee question like "Which coffee suits a marathon?" (`search_node` , then `no_grounding_node` , and no approval), and answer `no` once (`Not approved -- nothing was sent.`). Those are the README's 5 Test Cases, checked by hand.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — a visible path, `build_graph(checkpointer)`, and the 5 Test Cases

Story — `state.py` / `nodes.py` / `test_project3.py`: Basic works, but "what happened?" means scrolling through step lines, and the 5 Test Cases are checked by hand. **If not:** a change that broke the rejection path would only be noticed when a rejected answer went out anyway.

```
# state.py
import operator
from typing import Annotated, TypedDict


class ProjectState(TypedDict):
    task: str
    sources: list[dict]
    draft: str
    approved: bool
    answer: str
    # every node adds its own name, so the path taken is visible
    path: Annotated[list[str], operator.add]
```

```
# nodes.py
from langchain_openai import ChatOpenAI
from langgraph.types import interrupt

from kb import collection
from retriever import retrieve
from state import ProjectState

KEYWORDS = ["coffee", "brew", "grind", "water", "beans", "according to"]
MAX_DISTANCE = 1.2


def route_search(state: ProjectState) -> str:
    task = state["task"].lower()
    for keyword in KEYWORDS:
        if keyword in task:
            return "search"
    return "skip"
```

```
def search_node(state: ProjectState) -> dict:
    sources = retrieve(collection, state["task"])
    return {"sources": sources, "path": ["searched"]}


def best_distance(sources: list[dict]) -> float:
    best = sources[0]["score"]
    for source in sources:
        if source["score"] < best:
            best = source["score"]
    return best


def has_grounding(state: ProjectState) -> str:
    if not state["sources"]:
        return "no_grounding"
    if best_distance(state["sources"]) > MAX_DISTANCE:
        return "no_grounding"
    return "reason"


def reason_node(state: ProjectState) -> dict:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    if not state.get("sources"):
        draft = model.invoke(state["task"]).content
        return {"draft": draft, "path": ["answered directly"]}
    pieces: list[str] = []
    for source in state["sources"]:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = f"Context:\n{context}\n\nQuestion: {state['task']}"
    draft = model.invoke(prompt).content
    return {"draft": draft, "path": ["answered from documents"]}


def no_grounding_node(state: ProjectState) -> dict:
    return {
        "answer": "I don't have grounding for this in my documents.",
        "path": ["no grounding"],
    }


def approval_node(state: ProjectState) -> dict:
    decision = interrupt({
        "draft": state["draft"],
        "prompt": "Approve this answer before it's sent? (approved/rejected)",
```

```
    })
    approved = decision == "approved"
    if approved:
        return {"approved": True, "path": ["approved"]}
    return {"approved": False, "path": ["rejected"]}


def finish_node(state: ProjectState) -> dict:
    if state["approved"]:
        return {"answer": state["draft"]}
    return {"answer": "Not approved -- nothing was sent."}
```

```
# graph.py
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, StateGraph

from nodes import (
    approval_node,
    finish_node,
    has_grounding,
    no_grounding_node,
    reason_node,
    route_search,
    search_node,
)
from state import ProjectState


def build_graph(checkpointer=None):
    if checkpointer is None:
        checkpointer = MemorySaver()
    builder = StateGraph(ProjectState)
    builder.add_node("search_node", search_node)
    builder.add_node("reason_node", reason_node)
    builder.add_node("no_grounding_node", no_grounding_node)
    builder.add_node("approval_node", approval_node)
    builder.add_node("finish_node", finish_node)

    builder.set_conditional_entry_point(
        route_search, {"search": "search_node", "skip": "reason_node"}
    )
    builder.add_conditional_edges(
        "search_node", has_grounding,
        {"reason": "reason_node", "no_grounding": "no_grounding_node"},
    )
    builder.add_edge("reason_node", "approval_node")
```

```
    builder.add_edge("approval_node", "finish_node")
    builder.add_edge("no_grounding_node", END)
    builder.add_edge("finish_node", END)
    return builder.compile(checkpointer=checkpointer)
```

```
# main.py
from langgraph.types import Command

from graph import build_graph


def show_steps(graph, graph_input, config: dict) -> None:
    for update in graph.stream(graph_input, config, stream_mode="updates"):
        for node_name in update:
            print(f"  step: {node_name}")


def ask_reviewer(draft: str) -> str:
    print(f"Draft: {draft}")
    answer = input("Approve? (yes/no): ").strip().lower()
    if answer == "yes":
        return "approved"
    return "rejected"


def main() -> None:
    graph = build_graph()
    config = {"configurable": {"thread_id": "run-1"}}
    task = input("Your question: ")
    show_steps(graph, {"task": task, "path": []}, config)

    paused = graph.get_state(config)
    if paused.next:
        decision = ask_reviewer(paused.values["draft"])
        show_steps(graph, Command(resume=decision), config)

    final = graph.get_state(config).values
    print(f"Path:   {final['path']}")
    print(f"Answer: {final['answer']}")


if __name__ == "__main__":
    main()
```

```
# test_project3.py
# Runs the 5 Test Cases from the README. Each case prints OK or WRONG.
# Run it from inside this folder: python test_project3.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from langgraph.types import Command

from graph import build_graph


def run(graph, task: str, thread_id: str, decision: str | None) -> dict:
    config = {"configurable": {"thread_id": thread_id}}
    graph.invoke({"task": task, "path": []}, config)
    if decision is not None and graph.get_state(config).next:
        graph.invoke(Command(resume=decision), config)
    return graph.get_state(config).values


def check(case: str, passed: bool, path: list[str]) -> None:
    result = "OK"
    if not passed:
        result = "WRONG"
    print(f"{case} -> {result}: {path}")


if __name__ == "__main__":
    graph = build_graph()
    found = run(graph, "How hot should the water be?", "t1", "approved")
    check("case 1 (search, found)   ",
          found["path"][0] == "searched" and "approved" in found["path"],
          found["path"])

    direct = run(graph, "What is 2 + 2?", "t2", "approved")
    check("case 2 (no search)       ",
          "searched" not in direct["path"], direct["path"])

    nothing = run(graph, "Which coffee suits a marathon?", "t3", None)
    check("case 3 (nothing relevant)",
          nothing["path"] == ["searched", "no grounding"], nothing["path"])

    check("case 4 (approved)        ",
          found["answer"] == found["draft"], found["path"])

    rejected = run(graph, "How hot should the water be?", "t5", "rejected")
    check("case 5 (rejected)        ",
          rejected["answer"].startswith("Not approved"), rejected["path"])
```

Expected output of `python test_project3.py`:

```
case 1 (search, found)    -> OK: ['searched', 'answered from documents',
'approved']
case 2 (no search)        -> OK: ['answered directly', 'approved']
case 3 (nothing relevant) -> OK: ['searched', 'no grounding']
case 4 (approved)         -> OK: ['searched', 'answered from documents',
'approved']
case 5 (rejected)         -> OK: ['searched', 'answered from documents',
'rejected']
```

(Wrapped to fit the page.) Each node returns its own one-item `path` list, and `operator.add` joins them in order — Doc09's reducer.

Approach 2 — retry on search, and who approved when

Story: the exercises hardened each piece on its own; the Build Task's real question is whether they work together. **If not:** a brief search outage would fail the whole run, and the path couldn't say who approved.

Changes to Approach 1's files. **`state.py`** — one new field, `approved_by`:

```
# state.py
import operator
from typing import Annotated, TypedDict


class ProjectState(TypedDict):
    task: str
    sources: list[dict]
    draft: str
    approved: bool
    approved_by: str
    answer: str
    # every node adds its own name, so the path taken is visible
    path: Annotated[list[str], operator.add]
```

`nodes.py` — the Failure exercise's retry, and the Real-world exercise's who-and-when:

```
# nodes.py
import time

from langchain_openai import ChatOpenAI
from langgraph.types import interrupt
```

```
from kb import collection
from logging_setup import get_logger
from retriever import retrieve
from state import ProjectState
```

```
TRANSIENT_ERRORS = (TimeoutError, ConnectionError)
MAX_ATTEMPTS = 3
```

```
def search_with_retry(query: str) -> list[dict]:
    logger = get_logger(__name__)
    for attempt in range(MAX_ATTEMPTS):
        try:
            return retrieve(collection, query)
        except TRANSIENT_ERRORS as exc:
            if attempt == MAX_ATTEMPTS - 1:
                raise
            wait_seconds = 2 ** attempt
            logger.warning(f"search failed ({exc}); retry in {wait_seconds}s")
            time.sleep(wait_seconds)
    raise RuntimeError("search_with_retry left its loop unexpectedly")


def search_node(state: ProjectState) -> dict:
    sources = search_with_retry(state["task"])
    return {"sources": sources, "path": ["searched"]}
```

```
def approval_node(state: ProjectState) -> dict:
    decision = interrupt({
        "draft": state["draft"],
        "prompt": "Approve this answer before it's sent? (approved/rejected)",
    })
    who = decision["by"]
    if decision["decision"] == "approved":
        return {
            "approved": True,
            "approved_by": who,
            "path": [f"approved by {who} at {decision['at']}"],
        }
    return {
        "approved": False,
        "approved_by": who,
        "path": [f"rejected by {who} at {decision['at']}"],
    }
```

`main.py` — `ask_reviewer()` returns the who-and-when dict:

```
# main.py
from datetime import datetime, timezone

from langgraph.types import Command
```

```
def ask_reviewer(draft: str) -> dict:
    print(f"Draft: {draft}")
    reviewer = input("Your email: ").strip()
    answer = input("Approve? (yes/no): ").strip().lower()
    decision = "rejected"
    if answer == "yes":
        decision = "approved"
    return {
        "decision": decision,
        "by": reviewer,
        "at": datetime.now(timezone.utc).isoformat(),
    }
```

`test_project3.py` — `run()` sends the same kind of dict, and case 1 checks `found["approved"]`:

```
def run(graph, task: str, thread_id: str, decision: str | None) -> dict:
    config = {"configurable": {"thread_id": thread_id}}
    graph.invoke({"task": task, "path": []}, config)
    if decision is not None and graph.get_state(config).next:
        payload = {
            "decision": decision,
            "by": "tester@example.com",
            "at": datetime.now(timezone.utc).isoformat(),
        }
        graph.invoke(Command(resume=payload), config)
    return graph.get_state(config).values
```

Expected path for an approved run:

```
['searched', 'answered from documents', 'approved by ana@example.com at
2026-10-01T10:00:00+00:00']
```

Approach 3 — a Pydantic state instead of `TypedDict`

Story: `TypedDict` gives type hints but no checking — a node can write `{"approved": "yes"}` and nothing complains until something downstream breaks. **If not:** a wrong-typed field would surface as a confusing bug three nodes later.

Based on Approach 1. **`state.py`:**

```
# state.py
import operator
from typing import Annotated

from pydantic import BaseModel


class ProjectState(BaseModel):
    task: str
    sources: list[dict] = []
    draft: str = ""
    approved: bool = False
    answer: str = ""
    path: Annotated[list[str], operator.add] = []
```

`nodes.py` — the same functions, but a Pydantic state reaches each node as an **object**, so every `state["task"]` becomes `state.task` (`state["task"]` would raise `TypeError`):

```
# nodes.py
from langchain_openai import ChatOpenAI
from langgraph.types import interrupt

from kb import collection
from retriever import retrieve
from state import ProjectState

KEYWORDS = ["coffee", "brew", "grind", "water", "beans", "according to"]
MAX_DISTANCE = 1.2


def route_search(state: ProjectState) -> str:
    task = state.task.lower()
    for keyword in KEYWORDS:
        if keyword in task:
            return "search"
    return "skip"


def search_node(state: ProjectState) -> dict:
    sources = retrieve(collection, state.task)
```

```
    return {"sources": sources, "path": ["searched"]}


def best_distance(sources: list[dict]) -> float:
    best = sources[0]["score"]
    for source in sources:
        if source["score"] < best:
            best = source["score"]
    return best


def has_grounding(state: ProjectState) -> str:
    if not state.sources:
        return "no_grounding"
    if best_distance(state.sources) > MAX_DISTANCE:
        return "no_grounding"
    return "reason"


def reason_node(state: ProjectState) -> dict:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    if not state.sources:
        draft = model.invoke(state.task).content
        return {"draft": draft, "path": ["answered directly"]}
    pieces: list[str] = []
    for source in state.sources:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = f"Context:\n{context}\n\nQuestion: {state.task}"
    draft = model.invoke(prompt).content
    return {"draft": draft, "path": ["answered from documents"]}


def no_grounding_node(state: ProjectState) -> dict:
    return {
        "answer": "I don't have grounding for this in my documents.",
        "path": ["no grounding"],
    }


def approval_node(state: ProjectState) -> dict:
    decision = interrupt({
        "draft": state.draft,
        "prompt": "Approve this answer before it's sent? (approved/rejected)",
    })
    approved = decision == "approved"
    if approved:
```

```
        return {"approved": True, "path": ["approved"]}
    return {"approved": False, "path": ["rejected"]}


def finish_node(state: ProjectState) -> dict:
    if state.approved:
        return {"answer": state.draft}
    return {"answer": "Not approved -- nothing was sent."}
```

`graph.py`, `main.py` and `test_project3.py` don't change — `get_state(config).values` is still a dict, and the same 5 `OK` lines come back. Now a wrong type fails loudly: `graph.invoke({"task": 42}, config)` raises `pydantic.ValidationError` straight away, and a node that returned `{"approved": "not-a-bool"}` would raise one at the next step.

Difference from Approach 1, and between Approaches 2/3: Approach 1 makes the run visible and tested. Approach 2 adds a retry and a who-and-when record. Approach 3 changes no behavior — it checks every field's type at runtime.

Which one should you actually use?

Approach 1 meets every requirement — ship it first, with all 5 Test Cases passing. Add Approach 2's retry and audit once real users and real reviewers are involved, which is Project 3's whole point. Use Approach 3's Pydantic state once more than one person edits the graph, or a wrong-typed field has already bitten you. Project 3 builds the same graph step by step, in its own `src/` layout.